Kurs: „Kryptografia”

Wykład 6 i 7:
Realizacje algorytmów symetrycznych

Materiał obejmuje zakres programowy
2 godzin lekcyjnych

1. Systemy liczbowe

Systemy liczbowe jest to zbiór reguł do jednolitego i jednoznacznego zapisywania i reprezentowania liczb. Wykorzystywanym i ogólnie przyjętym do użytku współcześnie systemem liczbowym jest system dziesiętny, tzn. wykorzystywanych jest 10 znaków do zapisu liczb: 0,1,2,3,4,5,6,7,8,9. Przyrost rzędu liczby jest dodawany z lewej strony liczby, tak więc im dłuższa liczba w zapisie tym jest ona większa nominalnie. JeŜeli mamy liczbę 2178 to jest to 2*10^3 + 1*10^2 + 7*10^1 +8*10^0. Podstawą liczenia jest tu 10 (system dziesiętny) i jej kolejne potęgi określają, który jest to znak od prawej strony.

Złożone systemy można podzielić na addytywne oraz pozycyjne. Przykładem systemu addytywnego jest rzymski zapis liczb, tzn. dla mniejszych liczb przypisane są osobne znaki, a dla ich wielokrotności inne znaki. np. 17 = XVII, a 117 = CXVII. Systemy pozycyjne posiadają znaki tylko dla początkowych liczb. Dla większych liczb (większych od podstawy systemu) kaŜda nowa pozycja oznacza mnożnik potęgi podstawy systemu. Tak jak wcześniej pokazano przykładowa liczba 2178 jest reprezentowana w systemie pozycyjnym dziesiętnym (2*10^3 + 1*10^2 + 7*10^1 +8*10^0).

W informatyce wykorzystywany jest z powodu reprezentacji liczb w pamięci komputera system dwójkowy (binarny) tzn. istnieją dwa znaki do zapisu reprezentacji liczby: 0 oraz 1. Wartość każdego bitu jest reprezentowana w tym systemie. Jest to drugi najbardziej prymitywny z możliwych systemów, po systemie jedynkowym, w którym ilość zapisanych znaków oznacza liczbę, np. jeśli jedynym znakiem jest I, to liczbę 7 w systemie dziesiętnym można zapisać w systemie jedynkowym jako IIIIIII.
Aby zapisać liczbę podaną dziesiętnie w systemie dwójkowym należy przekształcić liczbę tak, aby każda pozycja reprezentowała potęgę liczby 2:

Dziesiętnie 19 = 16 + 0 + 0 + 2 + 1 = 1*2^4 + 0*2^3 + 0*2^2 + 1*2^1 + 1*2^0 = dwójkowo 10011.
Dziesiętnie 13 = 8 + 4 + 0 + 1 = 1*2^3 + 1*2^2 + 0*2^1 + 1*2^0 = dwójkowo 1101.

Analogicznie przekształcamy w drugą stronę:

Dwójkowo 110010 = 1*2^5 + 1*2^4 + 0*2^3 + 0*2^2 + 1*2^1 + 0*2^0 = 32+16+0+0+2+0=dz. 50.
Dwójkowo 11010 = 1*2^4 + 1*2^3 + 0*2^2 + 1*2^1 + 0*2^0 = 16+8+0+2+0=dz. 26.

Często w informatyce wykorzystywane są również systemy ósemkowy i szesnastkowy (bajt zajmuje 8 bitów). Rozważmy przykład systemu szesnastkowego (nazywany również jako hexadecimal - hex.). Mamy dany następujący zbiór znaków: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F. W sposób trywialny można przekształcić liczby w systemie dwójkowym na system szesnastkowy i odwrotnie. Każde 4 pozycje w systemie dwójkowym zamieniamy na system szesnastkowy od prawej strony:

Dwójkowo 10100010001111010001111010 = 10 1000 1000 1111 0100 0111 1010 = hex. 288F47A.
Dwójkowo 1101001110111010011 = 110 1001 1101 1101 0011 = hex. 69DD3.

Analogicznie przekształcamy w drugą stronę:

Hex. 57FDA = dwójkowo 101 0111 1111 1101 1010.
Hex. A902 = dwójkowo 1010 1001 0000 0011.

2. Szyfr symetryczny DES

DES jest szyfrem blokowym, tzn. operuje na blokach o określonej wielkości (64 bity) i zwraca szyfrogram o takiej samej wielkości. Każdy blok jest dzielony na dwa bloki o długości 32 bitów (lewa połowa oznaczona jako L(left), a prawa jako R (right)).

Oznaczmy wiadomość jako M (message) = 0123456789ABCDEF w zapisie szesnastkowym. Wiadomość zapisaną szesnastkowo rozpisujemy binarnie:

M = 0000 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100 1101 1110 1111
L = 0000 0001 0010 0011 0100 0101 0110 0111
R = 1000 1001 1010 1011 1100 1101 1110 1111

DES wykorzystuje klucz 56-bitowy dla bloków 64-bitowych. Klucz jest przechowywany na 64 bitach, jednak wykorzystywane są tylko bity poza każdym ósmym kolejnym bitem (8,16,32..).

Przyjmijmy klucz K (key):

K = 133457799BBCDFF1 , co binarnie daje
K = 00010011 00110100 01010111 01111001 10011011 10111100 11011111 11110001

2.1 Tworzenie 16 podkluczy długości 48 bitów.

Klucz K jest poddawany permutacji PC-1 określonej poniżej. Pierwszy element to 57, co oznacza, że 57 bit klucza staje się pierwszym, kolejny to 49, co oznacza, że 49 bit klucza staje się drugim itd. Należy zauważyć, że w K+ (kluczu po permutacji) występuje tylko 56 bitów.

PC-1

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

K = 00010011 00110100 01010111 01111001 10011011 10111100 11011111 11110001
K+ = 1111000 0110011 0010101 0101111 0101010 1011001 1001111 0001111

Klucz K+ dzielimy teraz na dwie części 28 bitowe C_0 oraz D_0.

C_0 = 1111000 0110011 0010101 0101111
D_0 = 0101010 1011001 1001111 0001111

Mając utworzone C_0 oraz D_0 tworzymy 16 bloków par C_n oraz D_n (1<=n<=16). Każda para bloków tworzona jest na podstawie poprzedniej pary (C_n oraz D_n na podstawie C_{n-1} oraz D_{n-1}) w ten sposób, że przesuwamy bity w lewo o określoną wartość podaną poniżej, a skrajny lewy bit staje się skrajnie prawym.

Numer iteracji	Liczba przesunięć w lewo
1	1
2	1
3	2
4	2
5	2
6	2
7	2
8	2
9	1
10	2
11	2
12	2
13	2
14	2
15	2
16	1

C_0 = 11110000110011001010101111
D_0 = 01010101011001100111100011111

C_1 = 11100001100110010101011111
D_1 = 10101010110011001111000111110

C_2 = 11000011001100101010111111
D_2 = 01010101100110011110001111101

C_3 = 00001100110010101010111111111
D_3 = 01010110011001111000111101101

C_4 = 00110011001010101011111111100
D_4 = 01011001100111100011110101101

C_5 = 11001100101010101111111110000
D_5 = 01100110011110001111010101101

C_6 = 00110010101010111111111000011
D_6 = 10011001111000111101010101101

C_7 = 11001010101011111111100001100
D_7 = 01100111100011110101010110110

C_8 = 00101010101111111110000110011
D_8 = 10011110001111010101011001101

C_9 = 01010101011111111100001100110
D_9 = 00111100011110101010110011101

C_{10} = 01010101111111110000110011001
D_{10} = 11110001111010101011001100110

C_{11} = 01010111111110000110011001010

D_{11} = 110001111010101010110011 0011

C_{12} = 010111111110000110011 0010101
D_{12} = 000111101010101011 0011001111

C_{13} = 011111111000011001 1001010101
D_{13} = 011110101010101100 1100111100

C_{14} = 111111100001100110 0101010101
D_{14} = 111010101010110011 0011110001

C_{15} = 111110000110011001 0101010111
D_{15} = 101010101011001100 1111000111

C_{16} = 111100001100110010 1010101111
D_{16} = 010101010110011001 1110001111

Teraz tworzymy klucze K_n (1<=n<=16), stosując permutację PC-2 do każdej pary C_n oraz D_n . Każda para ma długość wspólną 56 bitów, PC-2 wykorzystuje tylko 48 bitów.

PC-2

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

Pierwszy bit K_n jest 14 bitem C_nD_n ,drugi 17 bitem itd, kończąc na 48 bicie będącym 32 bitem C_nD_n.

Dla C_1D_1 = 1110000 1100110 0101010 1011111 1010101 0110011 0011110 0011110,
klucz K_1 = 000110 110000 001011 101111 111111 000111 000001 110010

Dla kolejnych kluczy:

K_2 = 011110 011010 111011 011001 110110 111100 100111 100101
K_3 = 010101 011111 110010 001010 010000 101100 111110 011001
K_4 = 011100 101010 110111 010110 110110 110011 010100 011101
K_5 = 011111 001110 110000 000111 111010 110101 001110 101000
K_6 = 011000 111010 010100 111110 010100 000111 101100 101111
K_7 = 111011 001000 010010 110111 111101 100001 100010 111100
K_8 = 111101 111000 101000 111010 110000 010011 101111 111011
K_9 = 111000 001101 101111 101011 111011 011110 011110 000001
K_{10} = 101100 011111 001101 000111 101110 100100 011001 001111
K_{11} = 001000 010101 111111 010011 110111 101101 001110 000110
K_{12} = 011101 010111 000111 110101 100101 000110 011111 101001
K_{13} = 100101 111100 010111 010001 111110 101011 101001 000001
K_{14} = 010111 110100 001110 110111 111100 101110 011100 111010
K_{15} = 101111 111001 000110 001101 001111 010011 111100 001010
K_{16} = 110010 110011 110110 001011 000011 100001 011111 110101

2.2 Kodowanie 64-bitowych bloków

Dane wejściowe, które mają być zaszyfrowane są rozmieszczone w blokach 64 bitowych. Rozważmy teraz naszą wiadomość o długości 64 bitów.
M = 0000 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100 1101 1110 1111
Pierwszą czynnością jaką należy wykonać jest poddanie permutacji IP bloku danych.

IP

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

58 bit wiadomości M staje się pierwszym bitem IP, 50 bit wiadomości M staje się drugim bitem IP, ostatecznie 7 bit wiadomości M staje się ostatnim bitem IP.

M = 0000 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100 1101 1110 1111
IP = 1100 1100 0000 0000 1100 1100 1111 1111 1111 0000 1010 1010 1111 0000 1010 1010

Dzielimy IP na lewą L_0 i prawą R_0 stronę po 32 bity.
Otrzymujemy:

L_0 = 1100 1100 0000 0000 1100 1100 1111 1111
R_0 = 1111 0000 1010 1010 1111 0000 1010 1010

Następnie wykonujemy 16 iteracji ($1<=n<=16$) wykorzystując funkcję f która działa na dwóch blokach , bloku danych 32 bitowym oraz bloku danych klucza K_n 42 bitowym i zwraca blok 32 bitowy. Wykonuje ona operację XOR na poszczególnych bitach.
Dla każdej iteracji n (1 do 16) wykonywane jest następujące przekształcenie:

$$\mathbf{L_n = R_{n-1}}$$
$$\mathbf{R_n = L_{n-1} + f(R_{n-1}, K_n)}$$

Rozpatrzmy n=1.

K_1 = 000110 110000 001011 101111 111111 000111 000001 110010
$L_1 = R_0$ = 1111 0000 1010 1010 1111 0000 1010 1010
$R_1 = L_0 + f(R_0, K_1)$

Zobaczmy teraz jak działa funkcja f dla naszego przykładu:
Najpierw należy rozszerzyć blok R_{n-1} z 32 bitów do 48 bitów. Wykorzystuje się do tego celu tablicę E-bit, która powtarza niektóre bity. Taka funkcja $E(R_{n-1})$ ma zadany blok 32 bitowy a zwraca 48 bitowy.

E-BIT

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Pierwszy bit $E(R_{n-1})$ jest 32 bitem R_{n-1}, drugi bit $E(R_{n-1})$ jest 1 bitem R_{n-1}, trzeci bit $E(R_{n-1})$ jest 2 bitem R_{n-1} itd.

Przykładowo dla R_0:

R_0 = 1111 0000 1010 1010 1111 0000 1010 1010
$E(R_0)$ = 011110 100001 010101 010101 011110 100001 010101 010101

Następnie w ramach funkcji f wykonujemy XOR dwóch bloków danych $E(R_{n-1})$ oraz K_n. Funkcję XOR będziemy oznaczać jako +.

K_1 = 000110 110000 001011 101111 111111 000111 000001 110010
$E(R_0)$ = 011110 100001 010101 010101 011110 100001 010101 010101
$K_1 + E(R_0)$ = 011000 010001 011110 111010 100001 100110 010100 100111

Rozszerzyliśmy R_{n-1} z 32 bitów do 48 bitów i rezultat poddaliśmy operacji XOR z kluczem K_n. Mamy teraz 8 grup 6 bitowych, które będą wykorzystywane jako adresy w tablicach 'S boxes'.
KaŜda grupa 6 bitów da adres w każdym 'S boxie'. Rezultatem wyliczenia S() będzie 4bitowa wartość, a parametrem 6 bitowa wartość.

Zapiszmy 48 bitowy wynik $K_n + E(R_{n-1})$ jako:

$$\mathbf{K_n + E(R_{n-1})=B_1B_2B_3B_4B_5B_6B_7B_8}$$

gdzie B_i jest grupą sześciu bitów.

Teraz obliczamy

$$\mathbf{S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8)}$$

gdzie $S_i(B_i)$ odnosi się do i-tego 'S boxa'.

Do obliczenia wartości $S_1()$ wykorzystywana jest następująca tabela:

S1

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

$S_1(B)$ obliczane jest w następujący sposób w oparciu o powyższą tabelę (B jest długości 6 bitów):

Pierwszy i ostatni bit wartości B zapisanej w systemie binarnym jako jedno daje liczbę od 00 do 11 (dziesiętnie od 0 do 3). Nazwijmy tę wartość jako **i**. Środkowe 4 bity wartości B dają liczbę od 0000 do 1111 (dziesiętnie od 0 do 15). Nazwijmy tę wartość jako **j**. Teraz znajdujemy w tabeli wartość w i-tym wierszu i j-tej kolumnie. Wynikiem jest wartość dziesiętna od o do 15 (binarnie od 0000 do 1111). Ta wartość jest wynikiem $S_1(B)$.

Dla przykładu weźmy B=011000, co odpowiada B_1 w obliczonym wcześniej $K_1 + E(R_0)$.Obliczamy więc $S_1(011000)$. Początkowy i końcowy bit daje 00, a środkowe 4 bity to 1100. Wartość i wynosi 0 (00), a wartość j wynosi 12 (1100). Tak więc wartość tę odczytujemy z tabeli w 0 wierszu i 12 kolumnie. Jest to wartość 5 (0101). Tak więc $S_1(011000)=0101$.

Poniżej przedstawiono tabele dla wszystkich ośmiu 'S boxów'.

S1

14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

S2

15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9

S3

10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12

S4

7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4

3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14

S5

2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3

S6

12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13

S7

4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12

S8

13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

Dla pierwszej iteracji obliczyliśmy:

$K_1 + E(R_0)$ = 011000 010001 011110 111010 100001 100110 010100 100111
$S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8)$ = 0101 1100 1000 0010 1011 0101 1001 0111

Ostatnim etapem obliczenia funkcji f jest wykonanie permutacji P obliczonych S-boxów.

f = P(S1(B1)S2(B2)...S8(B8))

Permutacja P jest zdefiniowana poniŜej:

P

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

Dla naszego przykładu:

$S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8)$ = 0101 1100 1000 0010 1011 0101 1001 0111

otrzymujemy:

f = 0010 0011 0100 1010 1010 1001 1011 1011

$R_1 = L_0 + f(R_0, K_1)$ =
=1100 1100 0000 0000 1100 1100 1111 1111
+0010 0011 0100 1010 1010 1001 1011 1011
=**1110 1111 0100 1010 0110 0101 0100 0100**

W kolejnej iteracji wyznaczymy:

$L_2 = R_1$
$R_2 = L_1 + f(R_1, K_2)$
....

i tak aż do uzyskania L_{16} oraz R_{16}.

Ostatecznie uzyskane L_{16} oraz R_{16} poddajemy permutacji **IP^{-1}** w odwróconej postaci $R_{16}L_{16}$. Pierwszy bit IP^{-1} jest 40 bitem $R_{16}L_{16}$,drugi bit IP^{-1} jest 8 bitem $R_{16}L_{16}$ itd.

IP^{-1}

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

Dla naszego przykładu:

L_{16} = 0100 0011 0100 0010 0011 0010 0011 0100
R_{16} = 0000 1010 0100 1100 1101 1001 1001 0101

zapisujemy:

$R_{16}L_{16}$ = 00001010 01001100 11011001 10010101 01000011 01000010 00110010 00110100
IP^{-1} = 10000101 11101000 00010011 01010100 00001111 00001010 10110100 00000101

co jest ostatecznym zaszyfrowanym wynikiem:

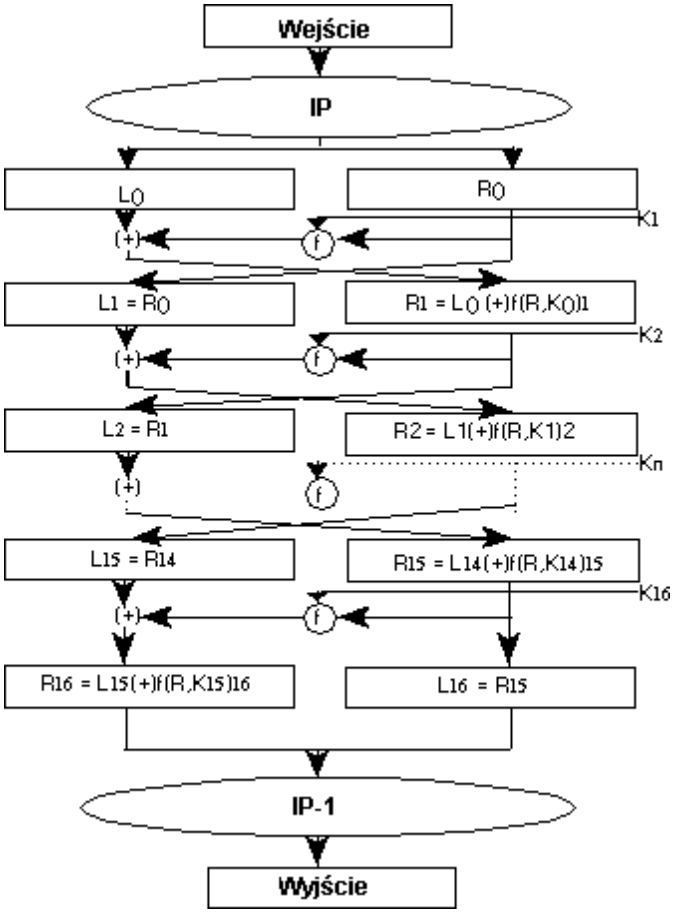
C(cipher) = 10000101 11101000 00010011 01010100 00001111 00001010 10110100
00000101
C = heksadecymalnie 85E813540F0AB405

Zaszyfrowana wiadomość M = 0123456789ABCDEF dla naszego klucza K wygląda zatem następująco:

C =85E813540F0AB405

2.3 Podsumowanie

Przedstawione obliczenia można zilustrować dodatkowo następującym diagramem:



Opisane szyfrowanie jest bardzo skomplikowane i niemal niewykonalne bez pomocy komputera dla dłuższych wiadomości z powodu ograniczeń czasowych i wydajnościowych. Deszyfrowanie jest operacją odwrotną do przedstawionej w przykładzie, w której szyfrowanie przebiega z odwróconą kolejnością w jakiej stosowane są klucze. Dzisiejsze technologie pozwalają na złamanie tego szyfru i nie jest on już uznawany za gwarancję bezpieczeństwa. Ulepszona wersja 3DES daje na dzień dzisiejszy gwarancję bezpieczeństwa, jednak w obliczu możliwości powstania komputera kwantowego także i ona jest mocno zagrożona.